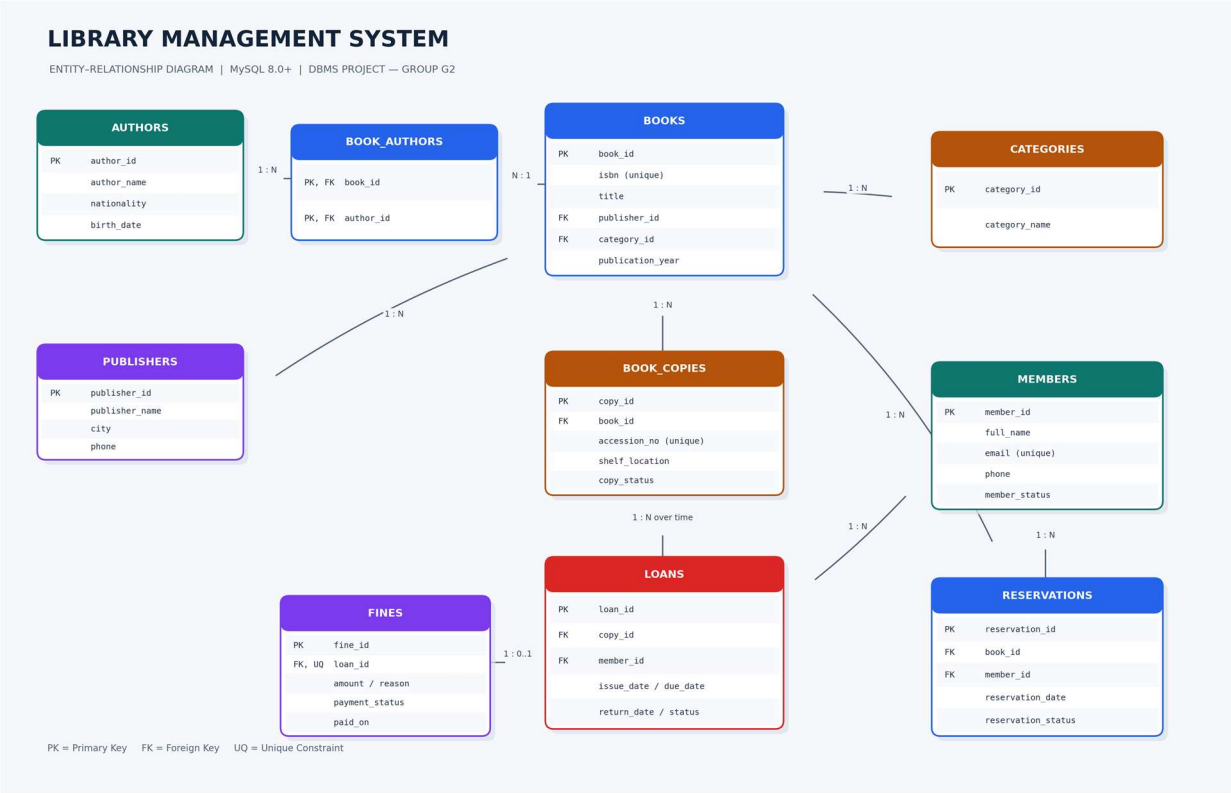
BRAINWARE UNIVERSITY

DATABASE MANAGEMENT SYSTEMS

PROJECT REPORT

LIBRARY MANAGEMENT SYSTEM



Project Group: G2
Submitted by: Raju Mondal , Ankush Mondal , Shamik Golder, Rana Khatua
Roll Number: BWU/BCC/25/007, BWU/BCC/25/008, BWU/BCC/25/010, BWU/BCC/25/010
Section: A
Faculty / Guide: MR. MAINUL HASAN & MR. INDRASIS DAS
Academic Year: 2026–2027

1. Executive Summary

The Library Management System (LMS) is a relational database solution for organizing a library's catalogue and circulation activities. It maintains bibliographic details, authors, publishers, categories, physical copies, members, loans, reservations, and fines. The database is designed to improve data consistency, make book availability easier to check, and support routine library reports.

2. Problem Statement

Develop a database-driven Library Management System for managing books, authors, physical book copies, members, book issue and return activities, reservations, and fines. The system should simplify library operations while maintaining accurate information regarding book availability and member transactions.

3. Objectives

- Create a central and structured catalogue of book titles and authors.
- Track each physical copy by a unique accession number and current status.
- Maintain member records and membership status.
- Record issue dates, due dates, returns, and loan status.
- Store reservation requests and support a waiting queue for titles.
- Record fines and payment status.
- Apply primary keys, foreign keys, unique constraints, checks, and indexes to support data integrity.
- Produce useful operational reports using SQL joins, aggregation, and filtering.

4. Scope and Assumptions

This submission implements the database layer using MySQL 8.0+. It covers catalogue maintenance, member records, physical-copy inventory, issue/return transactions, reservations, fines, and reports. A web or desktop interface, authentication, notifications, and automated fine workflows are future enhancements. Example people and transactions are fictional. The example fine of Rs. 5 per overdue day is illustrative and should be replaced with the institution's actual policy.

5. Technology Stack

Component	Technology / Purpose
Database	MySQL 8.0+
Database client	MySQL Workbench
Query language	SQL
Operating system	Windows / macOS / Linux
Application interface	Not included in this database prototype

6. Functional Modules

6.1 Catalogue Management

Stores book title, ISBN, edition, publication year, language, publisher, and category.

6.2 Author Management

Stores author details and supports the many-to-many relationship between authors and book titles.

6.3 Copy Inventory

Tracks each physical copy independently, including accession number, shelf location, and availability/status.

6.4 Member Management

Stores contact details, join date, and ACTIVE, SUSPENDED, or CLOSED status.

6.5 Circulation

Records loans, due dates, returns, and the staff member processing each transaction.

6.6 Reservations

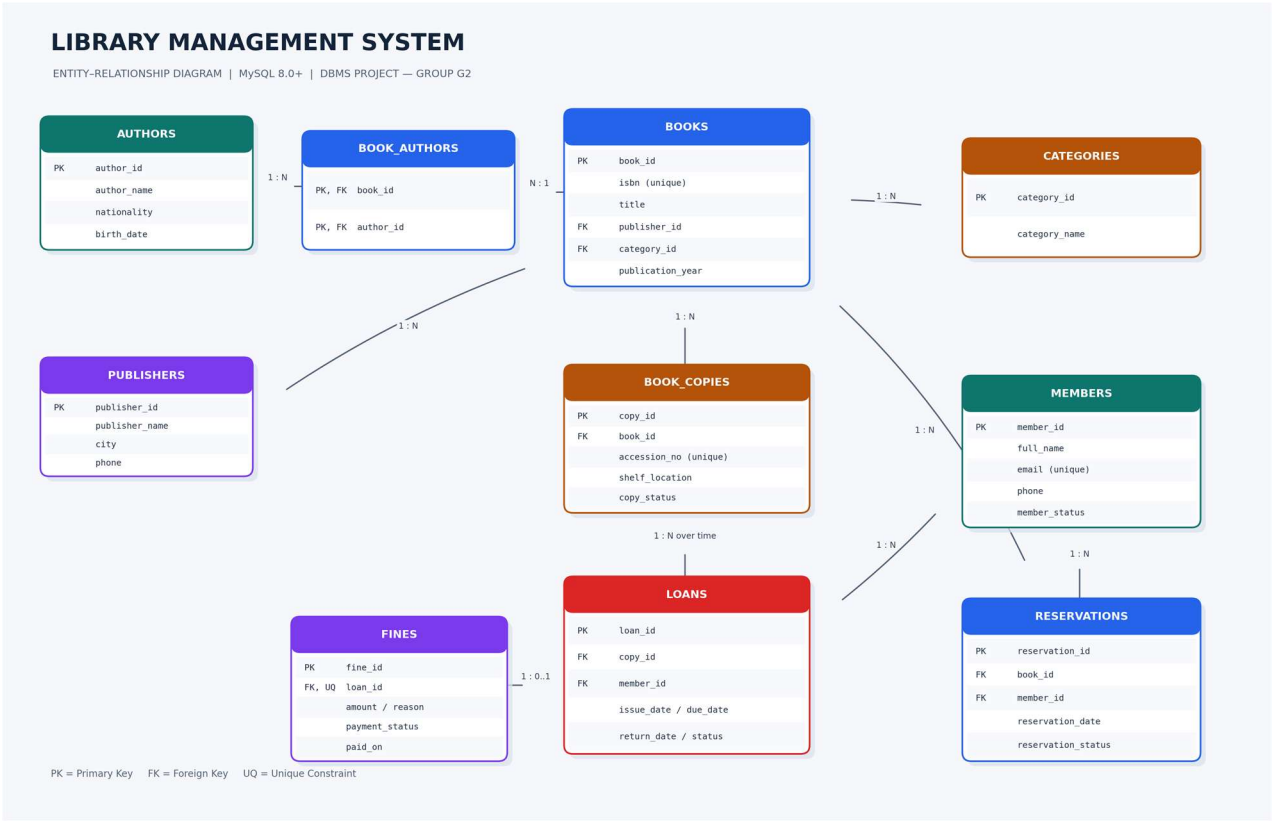
Maintains requests by member and title, including date and queue status.

6.7 Fine Management

Stores the fine amount, reason, assessment date, and payment state.

7. ER Diagram

The diagram below represents the entities and key relationships implemented by the SQL schema.



8. Database Schema

Table	Key attributes	Purpose
authors	author_id (PK), author_name, nationality,	Author information

	birth_date	
publishers	publisher_id (PK), publisher_name (UQ), city, phone	Publisher details
categories	category_id (PK), category_name (UQ)	Classification of titles
books	book_id (PK), isbn (UQ), title, publisher_id (FK), category_id (FK), publication_year, edition, language_name	Title-level catalogue record
book_authors	book_id (PK/FK), author_id (PK/FK)	Junction table for book-author M:N
members	member_id (PK), full_name, email (UQ), phone, address, joined_on, member_status	Member account record
book_copies	copy_id (PK), book_id (FK), accession_no (UQ), shelf_location, copy_status	Physical copy inventory
loans	loan_id (PK), copy_id (FK), member_id (FK), issue_date, due_date, return_date, loan_status	Issue/return transaction
reservations	reservation_id (PK), book_id (FK), member_id (FK), reservation_date, reservation_status, notes	Reservation queue
fines	fine_id (PK), loan_id (FK/UQ), amount, reason, assessed_on, payment_status, paid_on	Fine and payment record

9. Relationship Rules

- Publisher 1:M Books — each book title can reference one publisher; a publisher can publish many titles.
- Category 1:M Books — each title can be classified under a category.
- Books M:N Authors — implemented using book_authors.
- Book 1:M Book Copies — a title may have multiple physical copies.
- Member 1:M Loans — a member can have multiple loan transactions over time.
- Book Copy 1:M Loans over time — each copy can be borrowed repeatedly, but only one active loan is permitted per copy.
- Book 1:M Reservations and Member 1:M Reservations.
- Loan 1:0..1 Fine — the unique loan_id in fines permits at most one fine record per loan.

10. Normalization and Integrity

The design separates distinct entities to reduce duplication. Author data is separated from books and connected through book_authors; book titles are separated from individual physical copies; publishers and categories have their own tables. This structure supports Third Normal Form (3NF) when non-key attributes depend on the key, the whole key, and nothing but the key. Primary keys identify rows, foreign keys preserve referential integrity, unique constraints protect ISBN/email/accession numbers, and check constraints reject invalid due dates or negative fine amounts.

11. Important Business Rules

- A physical copy must have a unique accession number.
- A loan must refer to an existing member and existing copy.
- A due date cannot be earlier than the issue date.
- A fine amount cannot be negative.
- Before issuing, the application/operator must confirm the member is ACTIVE and the copy is AVAILABLE.
- When a copy is issued or returned, the copy status must be updated consistently with the loan transaction.

- The issue/return examples use transactions; production deployment should validate row counts and roll back if any step fails.

12. Sample SQL Reports

12.1 Book catalogue with category and publisher

```
SELECT b.title, b.isbn, c.category_name, p.publisher_name
FROM books b LEFT JOIN categories c ON c.category_id=b.category_id
LEFT JOIN publishers p ON p.publisher_id=b.publisher_id;
```

12.2 Available copies by title

```
SELECT b.title,
 SUM(CASE WHEN bc.copy_status='AVAILABLE' THEN 1 ELSE 0 END) AS available_copies
FROM books b LEFT JOIN book_copies bc ON bc.book_id=b.book_id
GROUP BY b.book_id,b.title;
```

12.3 Current loans

```
SELECT m.full_name,b.title,l.issue_date,l.due_date
FROM loans l JOIN members m ON m.member_id=l.member_id
JOIN book_copies bc ON bc.copy_id=l.copy_id
JOIN books b ON b.book_id=bc.book_id
WHERE l.return_date IS NULL;
```

12.4 Overdue loans

```
SELECT m.full_name,b.title,l.due_date,DATEDIFF(CURRENT_DATE,l.due_date) AS days_overdue
FROM loans l JOIN members m ON m.member_id=l.member_id
JOIN book_copies bc ON bc.copy_id=l.copy_id
JOIN books b ON b.book_id=bc.book_id
WHERE l.return_date IS NULL AND l.due_date<CURRENT_DATE;
```

12.5 Unpaid fines

```
SELECT m.full_name,b.title,f.amount,f.reason
FROM fines f JOIN loans l ON l.loan_id=f.loan_id
JOIN members m ON m.member_id=l.member_id
JOIN book_copies bc ON bc.copy_id=l.copy_id
JOIN books b ON b.book_id=bc.book_id
WHERE f.payment_status='UNPAID';
```

12.6 Category-wise title count

```
SELECT c.category_name,COUNT(b.book_id) AS total_titles
FROM categories c LEFT JOIN books b ON b.category_id=c.category_id
GROUP BY c.category_id,c.category_name;
```

13. Testing Plan

Test ID	Test case	Expected result
TC-01	Execute setup script	Database and all ten tables are created.
TC-02	Select catalogue report	Book titles and category/publisher data are returned.
TC-03	Check available copies	Counts reflect AVAILABLE copy status.
TC-04	Check current loans	Loans with no return date are listed.
TC-05	Insert duplicate accession number	Unique constraint rejects the duplicate.
TC-06	Insert loan with non-existent member	Foreign key rejects the invalid reference.
TC-07	Insert negative fine	Check constraint rejects the amount.

14. Installation and Execution

1. Install MySQL Server 8.0 or later and MySQL Workbench.
2. Open MySQL Workbench and connect to your local server.
3. Open library_management_system.sql in the SQL editor.
4. Execute the script from the beginning; it drops and recreates the database named library_management_system.
5. Refresh the SCHEMAS panel and expand the database tables.
6. Run each report query from the bottom of the script and inspect the result grid.
7. Capture genuine screenshots from your own execution and add them to the final report if required by your instructor.

15. Limitations

- No login, role-based access, or graphical interface is included.
- Copy-status updates are demonstrated as SQL workflows rather than enforced automatically by triggers.
- Fine assessment is stored as a record; a production system should implement the approved fine policy consistently.
- Concurrency and access permissions should be tested before deployment in a real library.

16. Future Enhancements

- Create a web application using Python Flask/Django or Java.
- Add librarian/member authentication and role-based permissions.
- Implement stored procedures or triggers for issue, return, and fine calculation.
- Add email/SMS reminders for due dates and reservation availability.
- Add barcode scanning, dashboards, audit logs, and backup/restore procedures.

17. Conclusion

The Library Management System demonstrates a relational database design for library catalogue and circulation operations. It applies normalization, primary and foreign keys, unique and check constraints, indexes, joins, aggregation, and transaction examples. The database can serve as the foundation for a larger application with a user interface and automated business workflows.

18. References

- Oracle, MySQL 8.0 Reference Manual, https://dev.mysql.com/doc/
- Elmasri, R. and Navathe, S. B., Fundamentals of Database Systems.
- Silberschatz, A., Korth, H. F., and Sudarshan, S., Database System Concepts.

19. Expanded SQL Query Bank — 200 Queries

This revised submission contains 200 individually numbered MySQL queries in the companion SQL script. The query bank progresses from basic SELECT, WHERE, ORDER BY and LIKE operations to multi-table JOINs, aggregate functions, GROUP BY, HAVING, nested queries, EXISTS, CASE expressions, CTEs, window functions and operational dashboards. All queries are written for the schema included in this project.

19.1 Query Coverage

Query range	Topics
Q001–Q020	Book catalogue, title search, language, edition and publication year
Q021–Q040	Authors, author-book mapping, publishers and categories
Q041–Q065	Member records, status, joining dates and member activity
Q066–Q090	Physical copy inventory, availability, shelf and status
Q091–Q120	Issue/return history, due dates, overdue loans and circulation statistics
Q121–Q140	Reservation queues, reservation status and demand
Q141–Q160	Fine records, payment status, totals and collection
Q161–Q180	Dashboards, popularity reports, eligibility and consistency checks
Q181–Q200	CTEs, window functions, UNION, CASE and overall statistics

19.2 Execution Notes

Run the setup and sample-data section first. Then execute the numbered queries individually in MySQL Workbench. The query bank is read-only and is safe to run against the supplied sample database. Some date-based reports use CURRENT_DATE, so their results change with the execution date. The supplied sample data is intentionally small; add more sample records to see richer aggregate and ranking results.

Important: the SQL setup starts by dropping and recreating the database named library_management_system. Back up any existing database with that name before running the script.